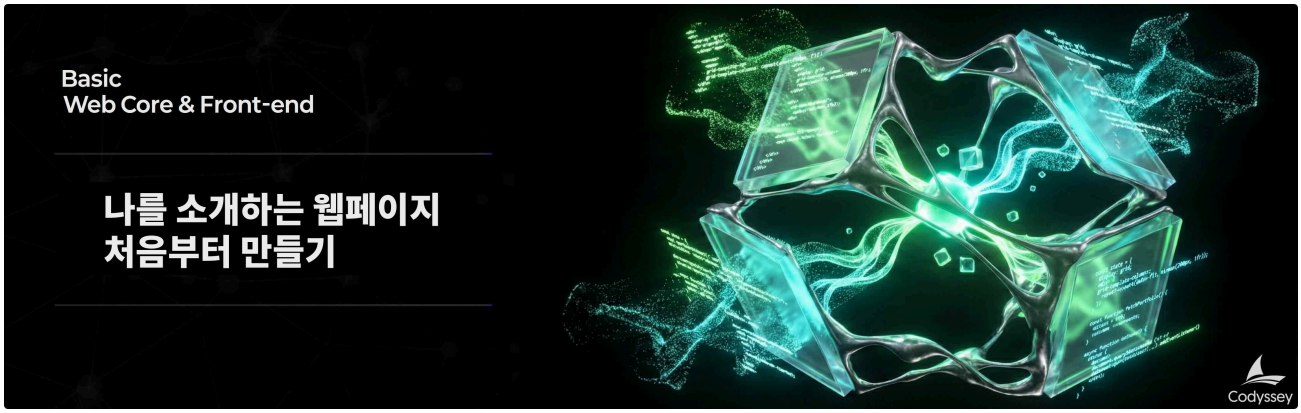
분야 AI/SW 기초 구분 웹 기초와 프론트엔드 학습시간 80시간 필수/선택 필수

나를 소개하는 웹페이지 처음부터 만들기

문제기술

기술적 설명



1. 미션 소개

HTML, CSS, JavaScript는 모든 웹 개발의 기초입니다. 브라우저가 이해하는 유일한 언어이며, React/Vue/Angular 같은 프레임워크도 결국 이 세 가지로 변환되어 동작합니다.

이 미션에서는 외부 라이브러리 없이 순수 HTML/CSS/JavaScript만으로 반응형 포트폴리오 웹사이트를 처음부터 끝까지 완성합니다. 단순히 화면을 그리는 것이 아니라, "사용자 이벤트 → DOM 조작 → 화면 변화"가 연결되는 웹의 동작 원리를 결과물로 확인합니다.

또한 GitHub API를 연동하여 실제 서비스에서 자주 등장하는 로딩/에러/빈 상태를 직접 처리하는 경험을 쌓습니다. 이 미션은 다음 미션인 React 학습의 필수 기반이 됩니다. React의 컴포넌트, 상태, 이벤트 개념은 모두 이 미션에서 다루는 DOM 조작과 이벤트 처리를 추상화한 것이기 때문입니다.

2. 최종 결과물

다음 조건을 만족하는 반응형 포트폴리오 웹사이트 1개를 완성한다.

1. 반응형 웹사이트

- 모바일, 태블릿, 데스크톱 등 모든 환경에서 레이아웃이 최적화되어 보인다.
- Hero, About, Skills, Projects, Contact, Footer 섹션을 포함한다.

2. 인터랙티브 UI

- 다크 모드 토글, 햄버거 메뉴, 부드러운 스크롤, 스크롤 애니메이션 등 사용자와 상호작용하는 기능이 동작한다.
- 폼 유효성 검사가 구현되어 있다.

3. 외부 API 연동

- GitHub API에서 본인의 저장소 목록을 가져와 Projects 섹션에 동적으로 렌더링한다.
- 로딩/에러/빈 상태가 UI로 표현된다.

4. 상태 유지

- 다크 모드 설정이 로컬스토리지에 저장되어, 새로고침 후에도 유지된다.

5. 배포

- GitHub Pages로 배포되어 외부에서 접속 가능한 URL이 존재한다.

3. 과제 목표

이 과제를 마친 후, 학습자는 아래를 스스로 설명할 수 있어야 한다.

- HTML에서 시맨틱 태그를 왜 사용하는지 또한 본인이 어떤 기준으로 구조를 설계했는지 설명할 수 있다.
- CSS에서 Flexbox와 Grid의 차이, 그리고 언제 각각을 선택해야 하는지 설명할 수 있다.
- `querySelector`로 DOM을 선택하고, `addEventListener`로 이벤트를 연결하는 흐름을 설명할 수 있다.
- 화살표 함수, 구조분해 할당, 배열 메서드(map/filter)가 왜 필요하고 어떻게 사용하는지 설명할 수 있다.
- `fetch`와 `async/await`로 비동기 데이터를 가져오고, 로딩/성공/실패 상태를 UI로 어떻게 표현했는지 설명할 수 있다.
- "하나의 기능"을 만들기 위해 이벤트 → 상태 변경 → DOM 업데이트가 어떻게 연결되는지 설명할 수 있다. (React의 상태-렌더링 흐름의 기초)

4. 기능 요구 사항

다음 요구사항을 모두 만족해야 한다.

1. 프로젝트 기본 구성
    - 프로젝트 폴더 구조가 최소한 다음 역할을 분리한다.
        - `index.html` (메인 페이지)
        - `css/` (스타일시트)
        - `js/` (JavaScript 파일)
        - `images/` (이미지 파일)
    - 외부 스타일시트와 JavaScript 파일을 HTML에 올바르게 연결한다.
    - VS Code + Live Server로 실시간 개발 환경을 구성한다.
2. HTML 구조 (시맨틱 마크업)
    - 전체 레이아웃을 `div`로만 감싸지 않고, 시맨틱 태그를 사용한다.
    - `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<footer>`
    - 페이지에 다음 섹션이 포함되어야 한다.
        - Hero (인사말, CTA 버튼)
        - About (자기소개, 프로필 이미지)
        - Skills (기술 스택 목록)
        - Projects (GitHub API 연동 카드)
        - Contact (문의 폼)
        - Footer (저작권, 소셜 링크)
    - 네비게이션에 각 섹션으로 이동하는 앵커 링크가 존재한다.
    - 모든 이미지에 의미있는 `alt` 속성이 있다.
    - 폼 요소에 `<label>`이 올바르게 연결되어 있다. (for-id 매칭)
3. CSS 스타일링 (레이아웃 & 반응형)
    - 외부 스타일시트(`css/style.css`)를 사용한다.
    - CSS 변수(`:root`)로 색상, 폰트, 간격을 정의한다.
    - 다크 모드용 CSS 변수를 별도로 정의한다. (`[data-theme="dark"]`)
    - 레이아웃 구현:
        - 네비게이션: Flexbox 사용 (로고 왼쪽, 메뉴 오른쪽)
        - Projects 카드: Grid 사용 (`auto-fit`, `minmax`로 반응형)
    - 반응형 디자인:
        - 모바일 퍼스트로 작성한다.
        - 브레이크포인트: 768px(태블릿), 1024px(데스크톱)
        - 모바일에서 네비게이션이 숨겨지고 햄버거 버튼이 나타난다.

- 시각 효과:
  - 버튼, 카드에 hover 효과 + transition 적용
  - 카드에 box-shadow 적용

4. JavaScript 기초 (DOM & 이벤트)

- JavaScript 파일을 `defer` 속성으로 연결한다.
- `var` 대신 `const`, `let`만 사용한다.
- HTML에 `onclick` 속성을 쓰지 않고, `addEventListener`로 이벤트를 연결한다.
- DOM 조작:
  - `querySelector`, `querySelectorAll`로 요소를 선택한다.
  - `textContent`, `innerHTML`로 내용을 변경한다.
  - `classList.add`, `remove`, `toggle`로 클래스를 조작한다.
- 이벤트 처리:
  - `click`, `submit`, `scroll`, `input` 이벤트를 다룬다.
  - `event.preventDefault()`로 기본 동작을 방지한다.

5. 인터랙션 구현

다음 인터랙션이 모두 동작해야 한다.

a. 햄버거 메뉴 토글

- 모바일에서 햄버거 버튼 클릭 시 메뉴가 나타난다.
- 다시 클릭하면 메뉴가 사라진다.
- `classList.toggle('active')` 활용

b. 부드러운 스크롤

- 네비게이션 메뉴 클릭 시 해당 섹션으로 부드럽게 이동한다.

c. 스크롤 탑 버튼

- 스크롤 300px 이상에서 버튼이 나타난다. (기준값은 자유 변경 가능하나 README에 명시)
- 클릭 시 페이지 맨 위로 이동한다.

d. 네비게이션 스타일 변경

- 스크롤 60px 이상에서 네비게이션 배경색이 변경된다. (기준값은 자유 변경 가능하나 README에 명시)

e. 다크 모드

- 토글 버튼 클릭 시 테마가 전환된다.
- 설정이 로컬스토리지에 저장되어 새로고침 후에도 유지된다.

f. 스크롤 애니메이션

- Intersection Observer 임계값(threshold)은 0.2 이상을 권장한다. (자유 변경 가능하나 README에 명시)

6. 폼 UX

- Contact 섹션에 문의 폼이 존재한다. (이름, 이메일, 메시지)
- 필수값 검증이 존재한다. (빈 필드 제출 불가)
- 이메일 형식 검증이 존재한다.
- 에러 메시지가 입력 필드 근처에 표시된다.
- 제출 시 `event.preventDefault()`로 기본 동작을 방지하고, 성공 메시지를 표시한다.

7. ES6+ 문법 & 배열 메서드

- 화살표 함수를 적절히 활용한다.
- 템플릿 리터럴로 HTML을 동적으로 생성한다.
- 구조분해 할당으로 객체/배열에서 값을 추출한다.
- 배열 메서드를 활용한다.
    - `map` : GitHub 데이터를 HTML 카드로 변환
    - `filter` : 특정 조건의 프로젝트만 표시 (선택)
    - `forEach` : 배열 순회

8. 비동기 처리 & API 연동
    - `fetch` 와 `async/await` 로 GitHub API를 호출한다.
        - 엔드포인트: `https://api.github.com/users/{본인아이디}/repos`
    - 다음 상태가 UI로 표현되어야 한다.
        - 로딩 상태: 데이터 요청 중 스피너 또는 "로딩 중..." 텍스트
        - 성공 상태: 카드 리스트 렌더링
        - 에러 상태: "프로젝트를 불러올 수 없습니다" 메시지 + 재시도 버튼
        - 빈 상태: "표시할 프로젝트가 없습니다" 메시지
    - `try/catch` 로 에러를 처리한다.
9. 상태 관리 패턴

    "사용자 이벤트 → 상태 변경 → 화면 업데이트" 흐름이 명확해야 한다.

    다음 3가지 이상의 "상태 → 렌더링" 흐름이 존재해야 한다.

    - 예시 1: 다크 모드 토글 → 테마 상태 변경 → 전체 화면 스타일 변경
    - 예시 2: API 호출 → 로딩/성공/에러 상태 변경 → Projects 섹션 렌더링 변경
    - 예시 3: 폼 입력 → 유효성 상태 변경 → 에러 메시지 표시/숨김
    - 예시 4: 필터 버튼 클릭 → 필터 상태 변경 → 프로젝트 목록 변경 (선택)
10. 배포
    * GitHub Pages로 배포한다.
    * 배포된 URL에서 모든 기능이 정상 동작해야 한다.
    * 반응형 레이아웃
    * 인터랙션 (햄버거 메뉴, 다크 모드, 스크롤 등)
    * GitHub API 연동
    * 폼 유효성 검사
    * README에 프로젝트 설명, 사용 기술, 배포 URL, 스크린샷이 포함되어야 한다.

5. 보너스 과제 (선택)

1. 프로젝트 필터링
    - GitHub 프로젝트를 언어별로 필터링하는 버튼을 만든다.
    - `array.filter()` 활용
2. 타이핑 효과
    - Hero 섹션에 타자기처럼 한 글자씩 나타나는 효과를 구현한다.
3. 폼 실제 전송
    - Formspree 또는 EmailJS를 연동하여 실제 이메일을 전송한다.
4. 시스템 다크 모드 감지

- `prefers-color-scheme` 미디어 쿼리로 시스템 설정을 감지한다.

개발환경

6. 개발 환경

- React, Vue, jQuery, Bootstrap, Tailwind CSS 등 외부 라이브러리 사용 금지
- 순수 HTML, CSS, JavaScript만 사용
- 아이콘(Font Awesome), 웹 폰트(Google Fonts)는 허용

제약조건

7. 제약 사항

- 핵심 목표:
  - UI 고퀄리티보다 "이벤트 → 상태 → 렌더링" 흐름 이해 우선
  - React 학습 전 필수 개념(DOM 조작, 이벤트, 비동기)을 체득하는 것이 목적
- 코드 스타일:
  - `var` 대신 `const`, `let` 사용
  - HTML에 `onclick` 대신 `addEventListener` 사용
  - 인라인 스타일(`style="..."`) 사용 금지
- 브라우저:
  - 최신 Chrome 브라우저에서 정상 동작
- 제출물:
  - GitHub 저장소 URL
  - 배포된 사이트 URL (GitHub Pages)
  - 데스크톱/모바일/다크모드 스크린샷
- GitHub API 주의사항:
  - 인증 없이 호출 시 시간당 60회 제한(레이트 리밋)이 있으므로, 짧은 시간 내 반복 새로고침을 피한다.
  - 레이트 리밋 발생 시(403 응답) 에러 상태 UI가 표시되도록 처리한다.

Test Case

8. 결과 예시

아래는 정답이 아니라 참고 예시다. 디자인은 본인의 개성에 맞게 자유롭게 구성한다.

[데스크톱 화면]

```
CSS
+-----------------------------------------------------------------+
| Logo                Menu                [D] | <- Header (Flex)
+-----------------------------------------------------------------+
|                                                 |
|                  Hello!                         |
|               I am [NAME].                      | <- Hero
```

```
|                                                                  |
|              [ View Projects ]   [ Contact ]                     |
|                                                                  |
+------------------------------------------------------------------+
| About Me                                                         |
|  +----------+  Intro text...                                     |
|  |  PHOTO   |  (skills / interests / short bio)                  |  <- About
|  +----------+                                                    |
+------------------------------------------------------------------+
| Projects (GitHub API)                                            |
|                                                                  |
|  +--------------+  +--------------+  +--------------+           |
|  | Repo 1       |  | Repo 2       |  | Repo 3       |           |
|  | Stars: 5     |  | Stars: 3     |  | Stars: 2     |           |
|  +--------------+  +--------------+  +--------------+           |
|                                                                  |
+------------------------------------------------------------------+
| Contact                                                          |
|  +------------------------------------------------------------+  |
|  | Name:    [_____________________]                           |  |
|  | Email:   [_____________________]                           |  |
|  | Message: [_____________________________________]           |  |
|  |                    [ Send ]                                |  |
|  +------------------------------------------------------------+  |
+------------------------------------------------------------------+
| (C) 2026 [NAME]   |  GitHub  |  LinkedIn                         |
+------------------------------------------------------------------+
```

[상태별 UI 예시]

- 로딩 중: Projects 섹션에 스피너가 보인다.
- 성공: 카드 리스트가 보인다.
- 에러: "프로젝트를 불러올 수 없습니다. [다시 시도]" 버튼이 보인다.
- 빈 데이터: "표시할 프로젝트가 없습니다."가 보인다.